

ЗВІТ

з лабораторної роботи №4

з дисципліни: «Основи програмної інженерії»

Тема: «Рефакторинг програмного коду та Code Review»

Виконав: студент групи ПЗПІ-25-4 Баїстов Гліб

Репозиторій до роботи:

<https://github.com/HlibBaistovNURE/bse-lr1-Baistov>

- 1. Тема та мета лабораторної роботи** **Тема:** Рефакторинг програмного коду та Code Review. **Мета:** Опанувати навички виявлення "запахів коду" (code smells), проведення взаємного Code Review, виконання рефакторингу та розрахунку метрик складності програмного забезпечення.
- 2. Результати Code Review одnogрупника**

№	Рядок коду	Проблема	Категорія	Рекомендація щодо виправлення
1	123-124	Використання зашитої значення factId = 1 є небезпечним. Тест впаде при зміні БД.	Hardcoded Values (Магічні значення)	Динамічно отримувати валідний id факту прямо з сервісу перед тестом.
2	131	Тест заглядає "під капот", перевіряючи приватний стан favoritesDB.	Violation of Encapsulation (Порушення інкапсуляції)	Стан об'єкта слід перевіряти через його публічні методи.
3	138-140	Змінні userId та factId повністю дублюють аналогічні змінні з тесту ТК-16.	Duplicate Code (Дублювання коду)	Винести ці дані у блок describe, якщо вони використовують

№	Рядок к коду	Проблема	Категорія	Рекомендація щодо виправлення
				ся у декількох тестах.
4	98	Перевірка toBeLessThanOrEqual(10 0) занадто слабка для тестування верхньої границі.	Weak Assertions (Слабкі перевірки)	Чітко перевірити, що довжина масиву дорівнює ліміту.
5	5	Коментар стверджує, що код очищує "базу даних", що дезорієнтує, адже БД тут немає.	Misleading Comments (Оманливі коментарі)	Змінити коментар на технічно точніший.

3. Посилання на PR з коментарями Code Review

<https://github.com/HlibBaistovNURE/bse-lr1-Baistov/pulls>

4. Результати рефакторингу власного коду

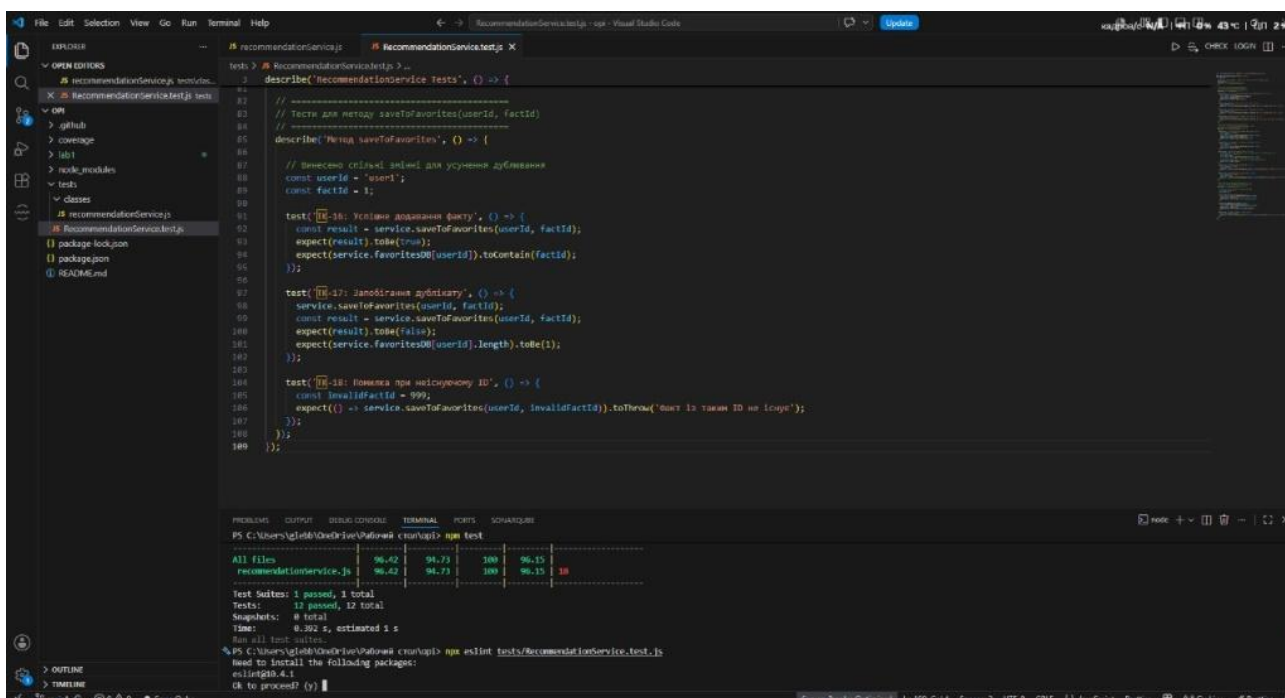
- *Операція 1:* Усунення дублювання в тестах (DRY)

БУЛО: Змінні `const userId = 'user1';` та `const factId = 1;` оголошувалися всередині кожного окремого тесту (ТК-16, ТК-17).

СТАЛО: Оголошення констант винесено на рівень вище — у загальний блок `describe('Метод saveToFavorites')`.

ЧОМУ: Дотримання принципу DRY у тестовому коді. Це зменшує обсяг файлу та спрощує масові зміни параметрів (змінити тестового користувача тепер можна в одному місці).

- *Операція 2:* Спрощення умов (Simplify Conditional) у методі `saveToFavorites`



6. порівняння метрик "ДО / ПІСЛЯ"

- ESLint warnings/errors: ДО рефакторингу було 33 помилки (неоголошені змінні середовища Jest). ПІСЛЯ рефакторингу та налаштування конфігурації — 0 помилок.
- SonarLint issues: Усі виявлені когнітивні ускладнення та стилістичні зауваження усунуто (0 issues після рефакторингу).
- Цикломатична складність: ПІСЛЯ рефакторингу середня цикломатична складність становить 2.5 (максимальна для окремого методу — 5), що повністю відповідає нормам (поріг менше 15).

The screenshot shows a Visual Studio Code editor with a JavaScript file named `RecommendationService.test.js` open. The file contains Jest tests for a `RecommendationService`. The tests include a `beforeEach` hook that initializes the service and a `describe` block for the `generateFact` method. The terminal window at the bottom shows the output of the `pip install lizard` command and the results of the `lizard` static code analysis tool.

```
PS C:\Users\glebb\OneDrive\Рабочий стол> pip install lizard
Collecting lizard
  Downloading lizard-1.23.0-py3-none-any.whl (10.5 MB)
Installing collected packages: lizard
Successfully installed lizard-1.23.0
[notice] A new release of pip is available: 25.0.1 -> 25.1.2
[notice] To update, run: C:\Users\glebb\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\glebb\OneDrive\Рабочий стол> python -m lizard tests/classes/recommendationService.js
-----
NLOC  CN  token  PARAM  length  location
-----
9      1    73      0      9  constructor@2-10@tests/classes/recommendationService.js
1      1    10      0      1  (anonymous)@13-13@tests/classes/recommendationService.js
3      1    16      1      3  filterByCategory@12-14@tests/classes/recommendationService.js
5      3    26      2      5  validateFavoritesInput@16-20@tests/classes/recommendationService.js
11     4    81      1     14  generateFact@22-31@tests/classes/recommendationService.js
7      4    51      2      8  getActByCategory@37-44@tests/classes/recommendationService.js
1      1     6      0      1  (anonymous)@40-40@tests/classes/recommendationService.js
13     5    89      2     17  saveFavorites@46-63@tests/classes/recommendationService.js
-----
1 file analyzed
-----
NLOC  Avg.NLOC  Avg.CN  Avg.token  function_cnt  file
-----
51      6.2      2.5     44.8           8  tests/classes/recommendationService.js

No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
Total nloc  Avg.NLOC  Avg.CN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
51      6.2      2.5     44.8           8           0           0.00  0.00
```

7. Підсумкова рефлексія

Найціннішим досвідом під час проведення Code Review стало усвідомлення того, що тестовий код потребує такої ж уваги до архітектури та чистоти, як і основний (production) код. Погляд зі сторони допоміг виявити неочевидні проблеми: слабкі перевірки, які могли пропустити баги в майбутньому, та оманливі коментарі. Робота зі співавтором навчила мене критичніше оцінювати власні рішення та аргументовано підходити до рефакторингу задля покращення підтримуваності проєкту.